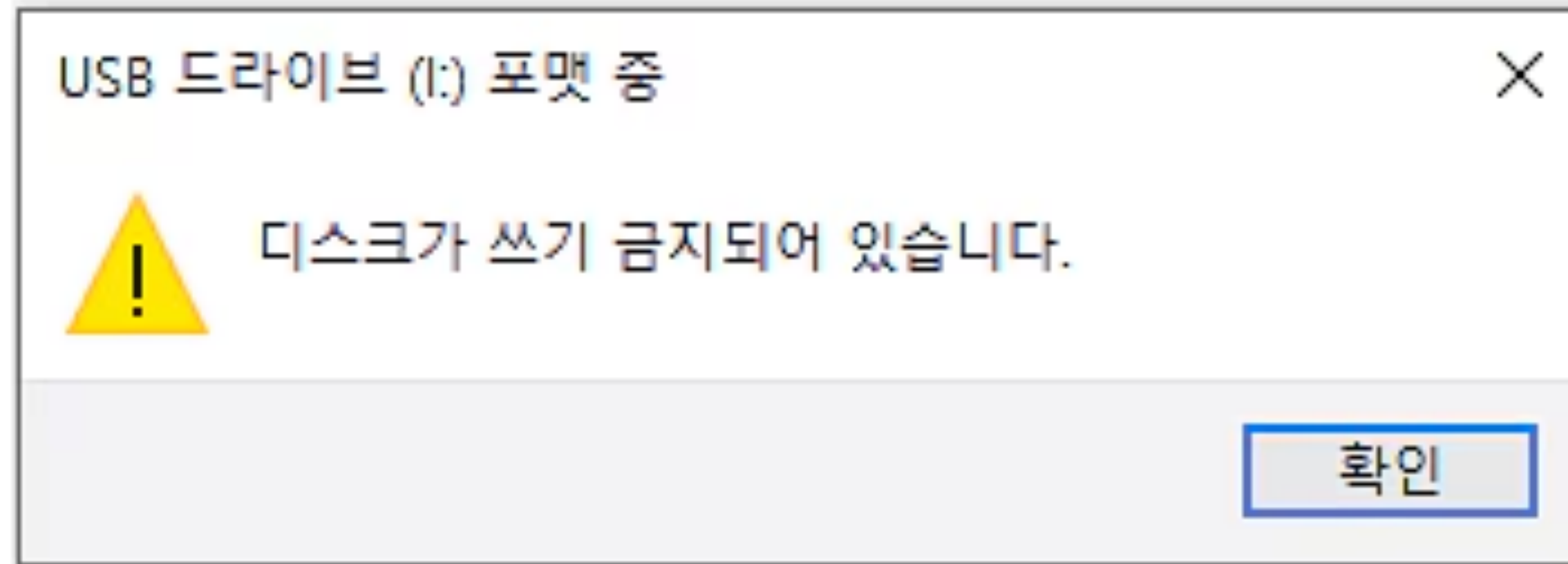




메시지 저장 구조 설계

메시지 브로커(MMMQ) 개발기 4

모코

목차

- 느린 소비자 문제
- 메시지 유실 문제

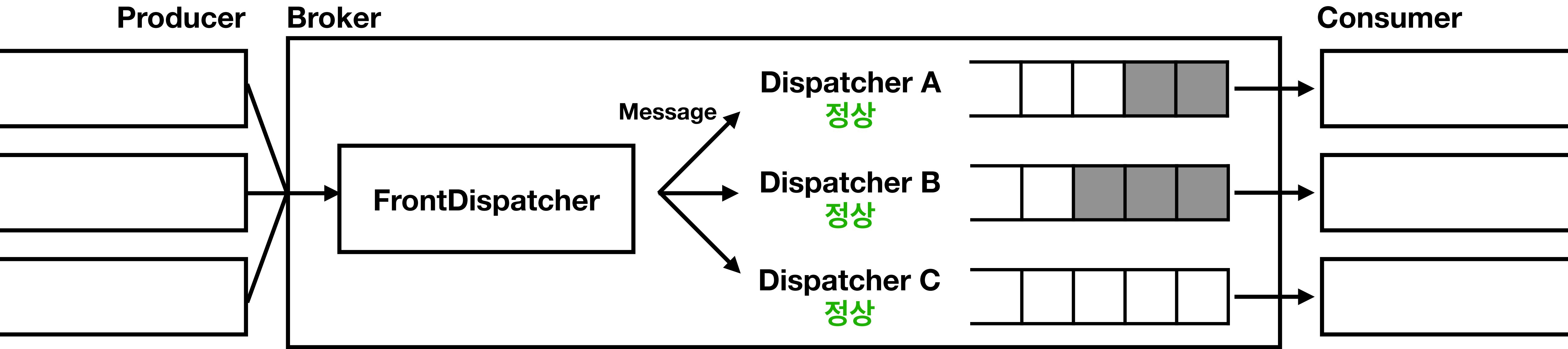
느린 소비자 문제

- 배경
- 데이터 구조

느린 소비자 문제

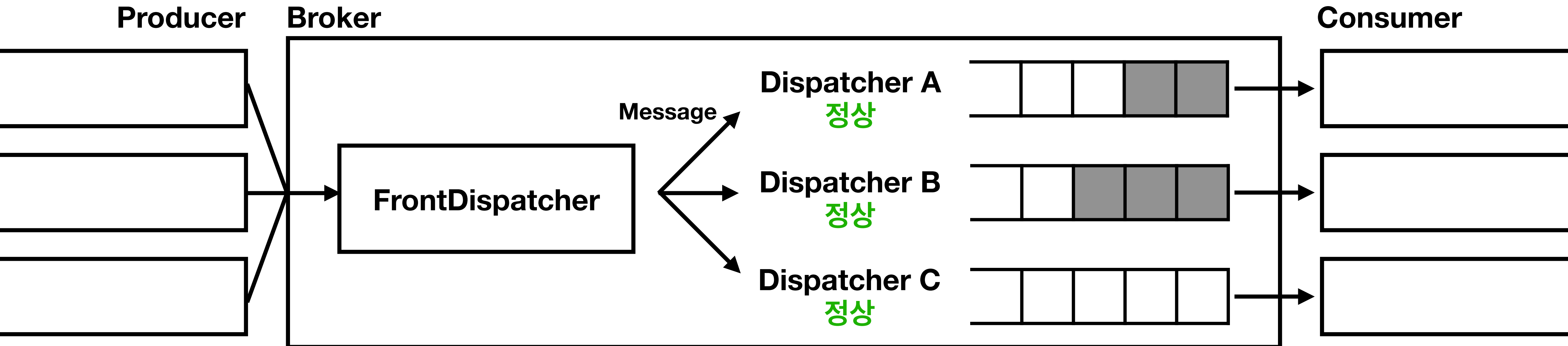
- 배경
- 데이터 구조

배경



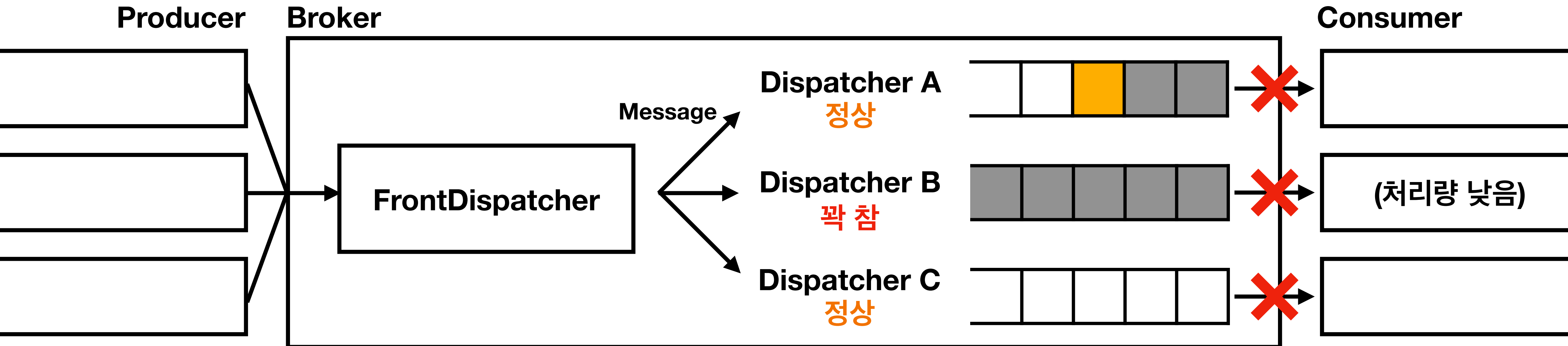
배경

- Dispatcher마다 독립적인 메시지 큐 관리



배경

- Dispatcher마다 독립적인 메시지 큐 관리
- 일부 Dispatcher의 큐가 꽉찬 경우 모든 Dispatcher 일괄 실패
 - (문제 1) 다른 Dispatcher까지 **메시지 전송 지연**
 - (문제 2) 이미 큐에 삽입한 Dispatcher들까지 **롤백 필요**(원자성 보장)



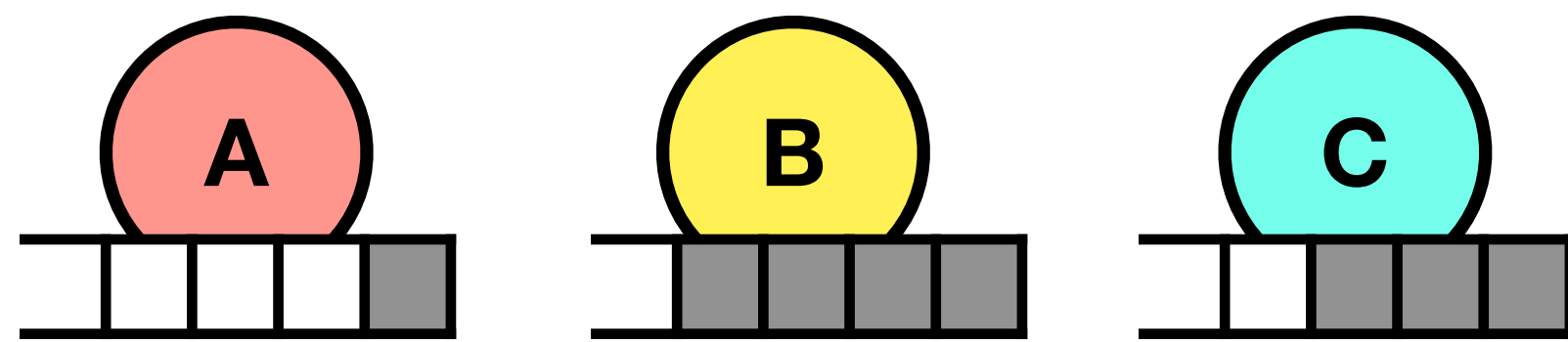
느린 소비자 문제

- 배경
- 데이터 구조

데이터 구조

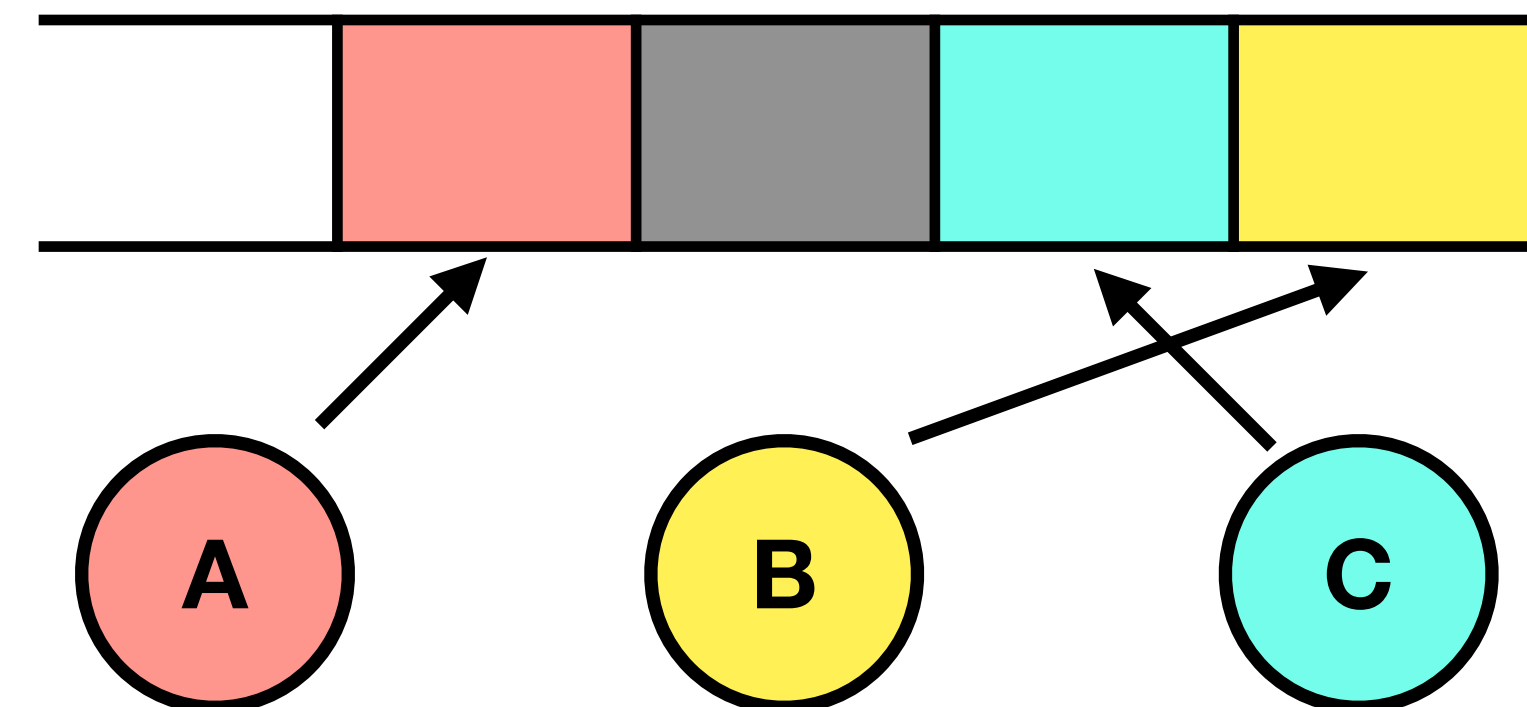
Before: 개별 큐

- Dispatcher 별 독립 큐
- 메시지를 Dispatcher 수만큼 복제
- 큐 하나가 꽉 차면 전체 정지
→ 느린 소비자의 처리량에 종속



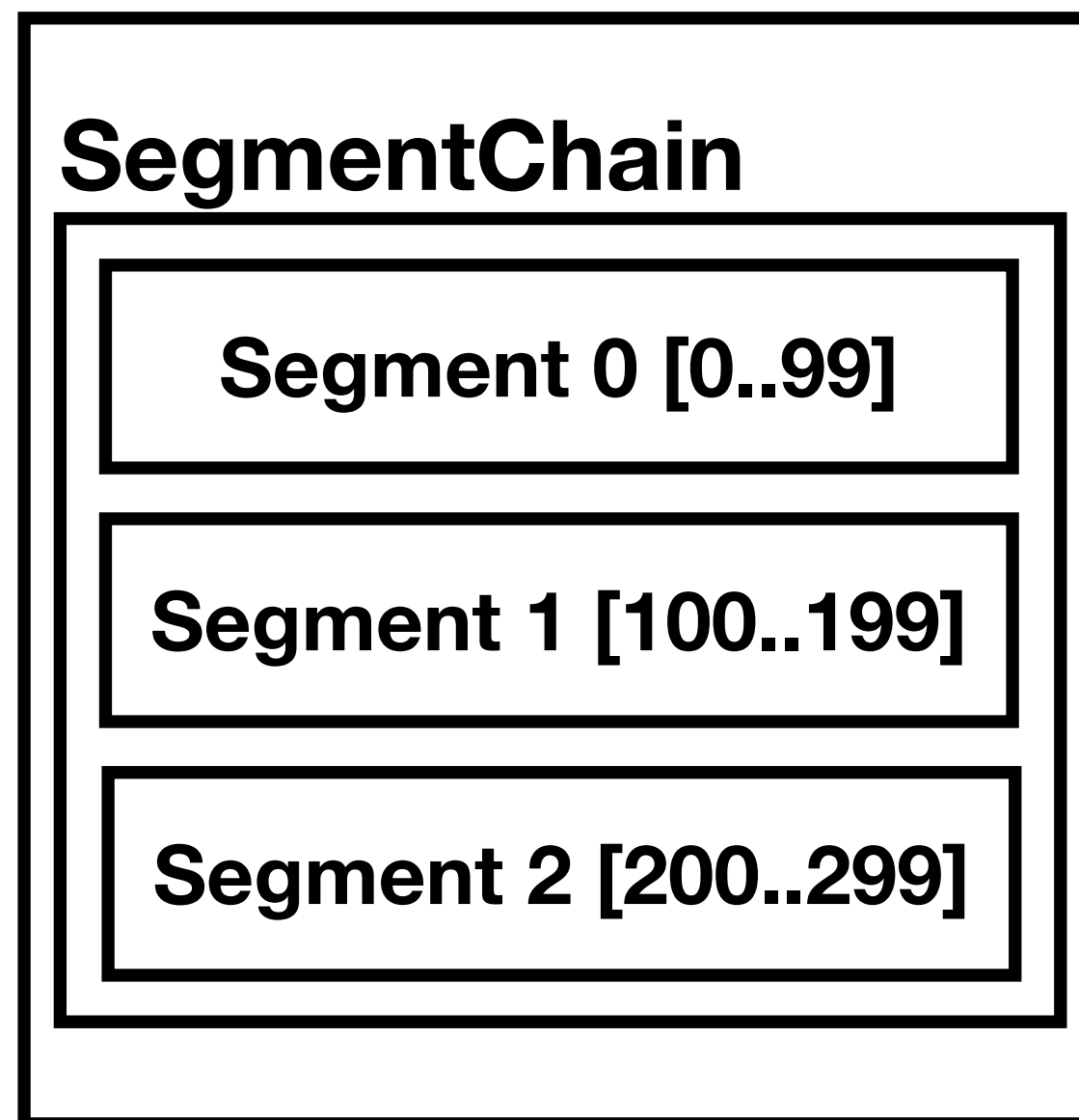
After: 중앙 로그 + 개별 오프셋

- Topic 별 로그
- Dispatcher마다 오프셋을 가지고 읽음
- 느린 소비자는 천천히,
빠른 소비자는 빠르게 처리 가능



데이터 구조

TopicQueue



- 소비자가 메시지를 소비해도 메시지가 삭제되지 않음
 - 메모리에 점점 누적
- 세그먼트 용량이 임계치를 넘어가면 세그먼트 분리
 - 더이상 참조되지 않는 오래된 세그먼트 제거

$\text{message-offset} = \text{segment-index} * \text{segment-size} + \text{inner-index}$

ex) segment-size = 100인 경우, offset 250 -> segment 2[50]

데이터 유실 문제

- 배경
- 영속화 모델
- 데이터 구조

데이터 유실 문제

- 배경
- 영속화 모델
- 데이터 구조

배경

- 기존에는 데이터를 메모리에서만 관리 → **재시작 시 전부 소멸**
 - 아직 보내지 못한 메시지 유실
 - Dispatcher 오프셋 유실
- 영속화 필요

데이터 유실 문제

- 배경
- **영속화 모델**
- 데이터 구조

영속화 모델

Before

- 메시지를 인메모리 배열에서 보관
 - 재시작 시 메시지 유실
- 오프셋을 인메모리 변수로 관리
 - Dispatcher 별 오프셋 유실
- ACK 시점: 인메모리 배열에 메시지 기입 후

After

- 메시지 디스크 영속화
 - 재시작 시 메시지 복구 가능
- 오프셋 디스크 영속화
 - 재시작 시 Dispatcher 별 오프셋 복구 가능
- ACK 시점: 디스크에 메시지 영속화 후

영속화 모델

커밋 시점

- 데이터 파일 append & fsync → 인덱스 파일 append & fsync
- 커밋 시점: 인덱스 fsync 완료 시점

데이터 파일

인덱스 파일

✓ commit point

영속화 모델

재시작 시 데이터 복구

- 브로커 재시작 시 데이터 & 인덱스 파일 기반으로 데이터 복구
- 데이터 & 인덱스 불일치 시 **인덱스를 신뢰**(SSOT, Single Source of Truth)

데이터 1
데이터 2
데이터 3
데이터 4

인덱스 1
인덱스 2
인덱스 3

데이터 4 제거

데이터 1
데이터 2
데이터 3
데이터 4

인덱스 1
인덱스 2
인덱스 3
인덱스 4
인덱스 5

부팅 중단

일반적인 상황에서는 발생 X

데이터 유실 문제

- 배경
- 영속화 모델
- 데이터 구조

데이터 구조

메시지 엔트리

length 4 bytes	CRC32C 4 bytes	message 가변 길이
--------------------------	--------------------------	-------------------------

데이터 구조

세그먼트 파일(.mmm)

- 엔트리들을 세그먼트 파일에 적재(append only)
- 데이터는 여러 개의 세그먼트 파일로 분리하여 영속화
- append 전 tail 세그먼트 용량이 임계치를 넘었는지 검사
 - 넘었으면 새로운 세그먼트 파일을 만들어 append

SegmentChain

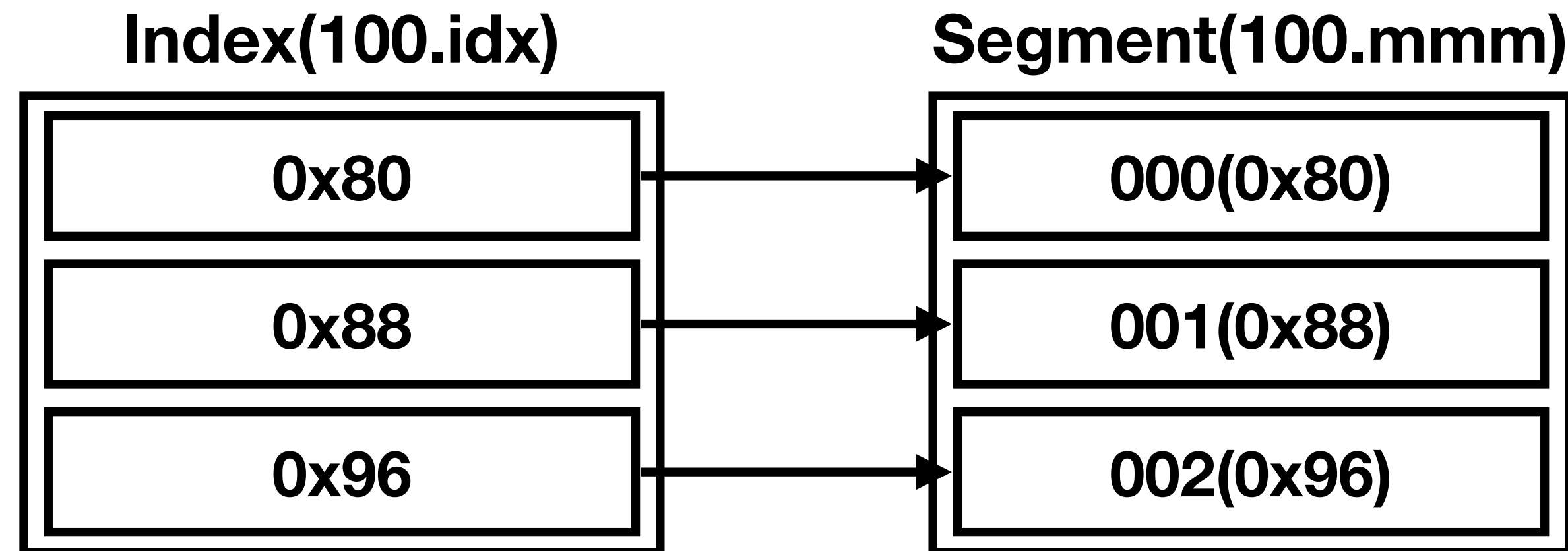


파일명: 첫 엔트리 오프셋 번호

데이터 구조

인덱스 파일(.idx)

- 메시지는 가변 길이이기 때문에 빠른 탐색을 위해 인덱스 필요
- 세그먼트 파일의 각 엔트리 시작 주소를 인덱스 파일에 적재(append only)
- 세그먼트 파일 분리 시 함께 분리

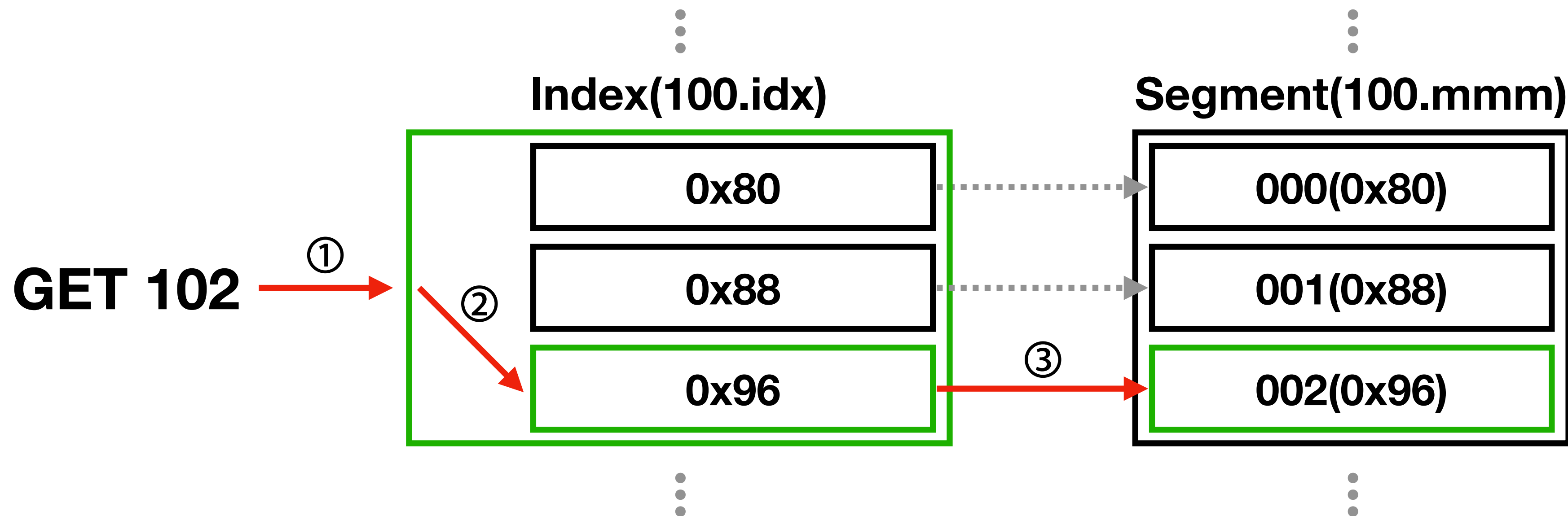


데이터 구조

데이터 조회 흐름

102번 데이터 조회 시도

1. Concurrent**SkipList**Map<Long, SegmentFile>#**floorKey()**로 대상 파일 탐색(100.idx)
2. **인덱스 파일**의 x번째 요소가 가진 **주소값 조회**($x=102_{\text{target_offset}} - 100_{\text{file_start_offset}}$)
3. 주소값으로 **세그먼트 파일**의 **실제 데이터 조회**



데이터 구조

체크포인트 파일(.checkpoint)

- Dispatcher 오프셋을 영속화
- **Consumer offset commit** 시 해당 오프셋을 체크포인트 파일에 기록(modify)
- Dispatcher가 새로운 토픽을 구독할 때마다 파일 추가

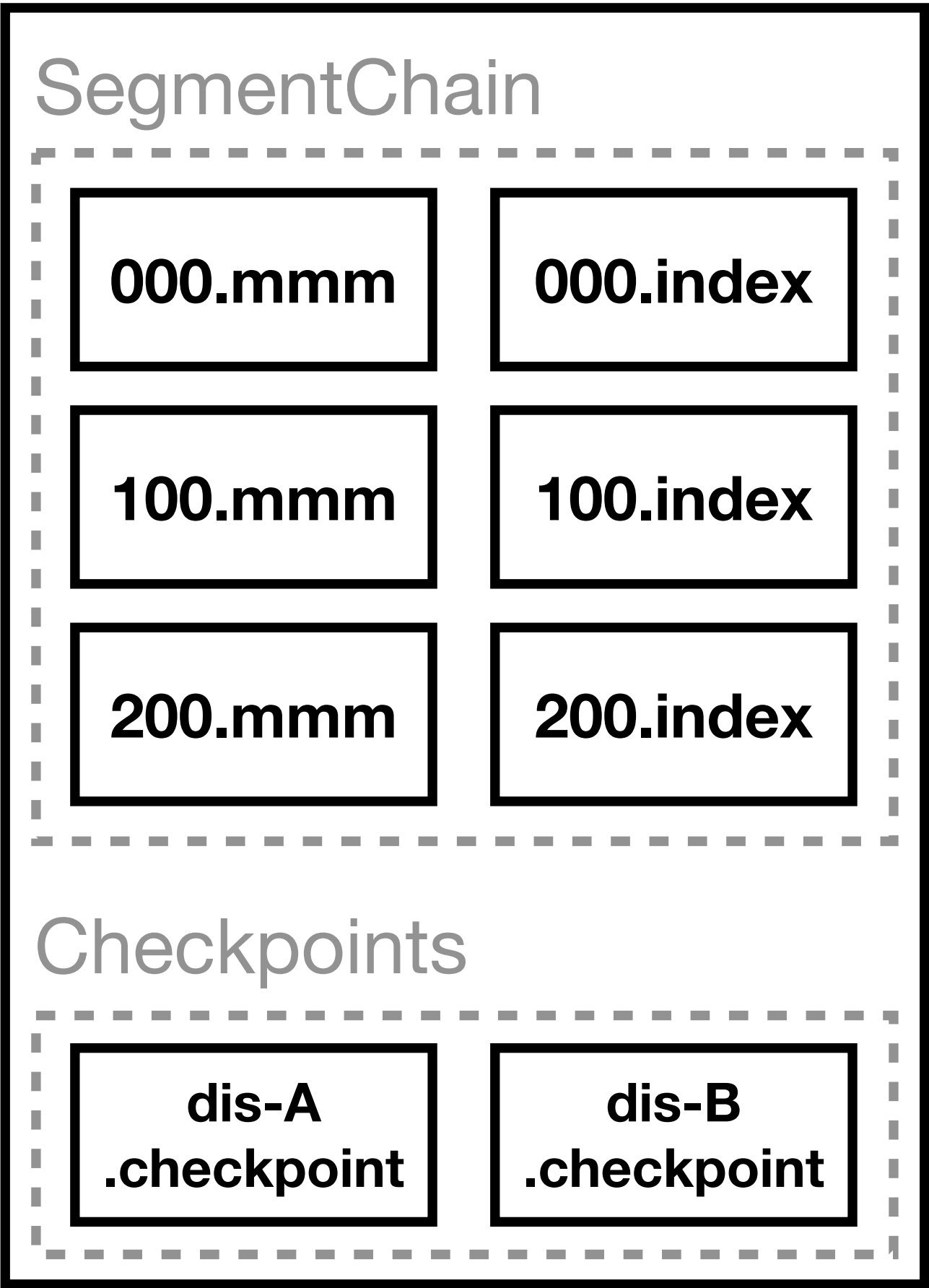
dispatcherA.checkpoint

230

데이터 구조

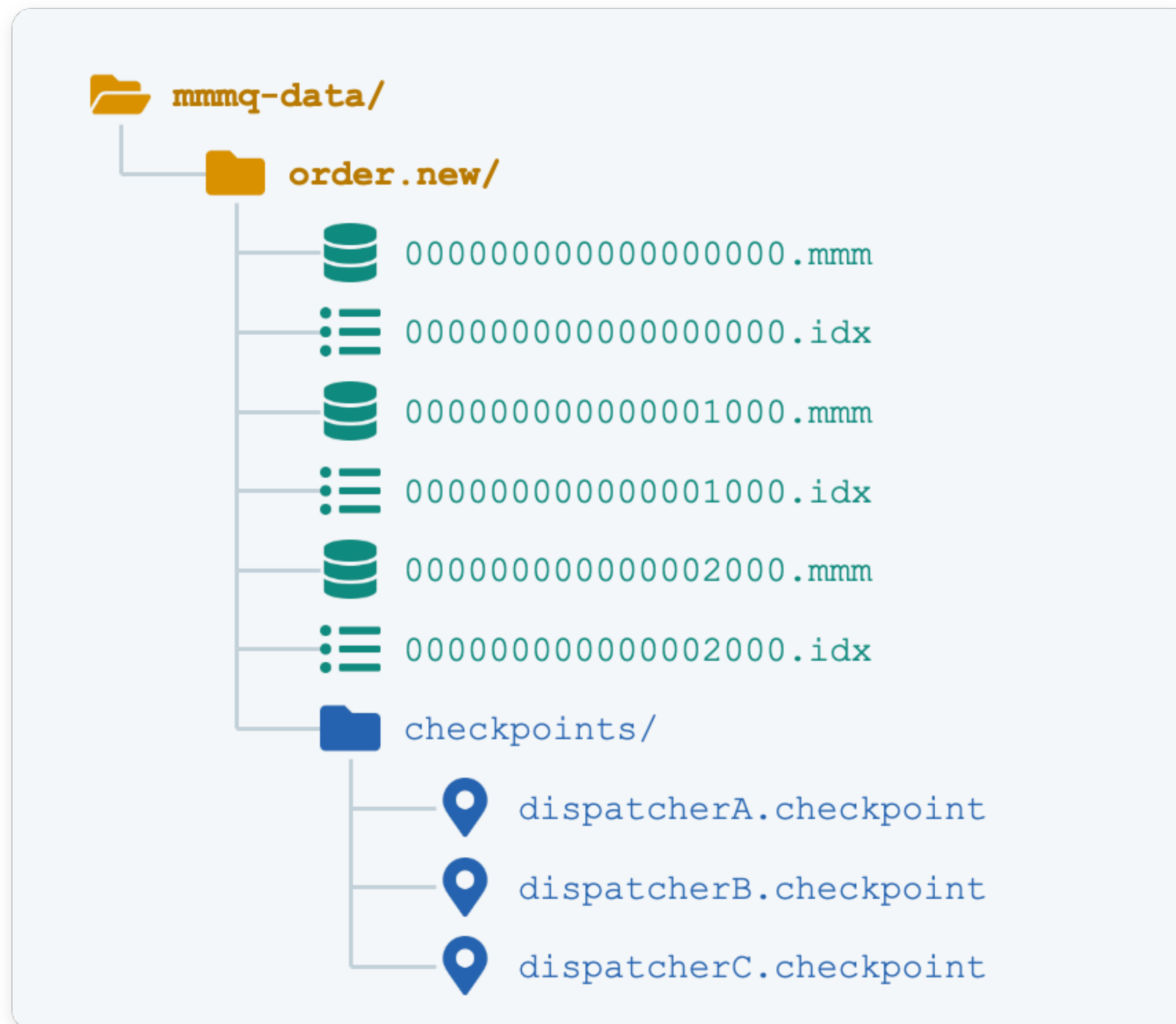
구조도

Topic



데이터 구조

디렉터리 구조



정리

정리

- 느린 소비자 문제
 - Dispatcher별 큐 구조 → **가장 느린 Dispatcher의 처리량에 종속**되는 문제 발생
 - 모든 Dispatcher가 하나의 로그를 바라보도록 **중앙화**하여 해결
- 데이터 유실 문제
 - 브로커 재시작 시 **메시지와 오프셋 유실**
 - 데이터를 여러 파일로 나눠 디스크 **영속화**하여 해결

감사합니다

MMMQ: <https://github.com/moko-meringue/mmmq>

1부 PR: <https://github.com/moko-meringue/mmmq/pull/5>

2부 PR: <https://github.com/moko-meringue/mmmq/pull/11>